PROYECTO FINAL · MARKETPLACE

🎸 Segunda Nota

Marketplace para comprar y vender instrumentos musicales usados, directo entre músicos.

Hito 1: Diseño y prototipo

1. Boceto de las vistas
2. Navegación entre vistas (públicas y privadas)
3. Dependencias del proyecto
4. Tablas de la base de datos y relaciones
5. Contrato de datos de la API REST

Integrante: Ignacio Robles

1. Boceto de las vistas

Página principal `/` pública

Logo [Iniciar sesión] [Regístrate]

Hero: imagen de fondo + título "Tu próximo instrumento ya tiene historia" + botones CTA

Guitarras | Bajos | Baterías | Teclados | ...

Card | Card | Card | Card

Footer

Registro de usuarios `/registro` pública

Logo [Iniciar sesión] [Regístrate]

Segunda Nota · Crear una cuenta

Nombre

Email

Contraseña | Repetir

URL foto (opcional)

Registrarme | Volver

Inicio de sesión `/login` pública

Logo [Iniciar sesión] [Regístrate]

Segunda Nota · Iniciar sesión

Email

Contraseña

Iniciar sesión | Volver

¿No tienes cuenta? Regístrate

Mi perfil `/perfil` privada

Logo · Tienda · Vender (foto) Nombre [Cerrar sesión]

(foto) Nombre

Mi perfil
Mis publicaciones
Mis favoritos
+ Nueva publicación

"¡Hola, Camila!"

Datos: nombre, email, miembro desde

N° publicaciones | N° favoritos

+ Publicar

Formulario crear publicación `/perfil/nueva` privada

Logo · Tienda · Vender (foto) Nombre [Cerrar sesión]

Menú perfil

Título

Categoría | Estado

Precio $ | Ubicación

URL imagen

Descripción

Publicar

Vista previa de la card en vivo

Galería de publicaciones `/tienda` privada

Logo · Tienda · Vender (foto) Nombre [Cerrar sesión]

Buscar por nombre | Categoría | Ordenar por

Imagen Título $ precio ♡ | Imagen Título $ precio ♡ | Imagen Título $ precio ♡ | Imagen Título $ precio ♡

Card | Card | Card | Card

Detalle de publicación `/publicacion/:id` privada

Logo · Tienda · Vender (foto) Nombre [Cerrar sesión]

Imagen de la publicación

Categoría · Estado

Título + $ precio

Descripción

Mensaje al vendedor | ♡

Vendedor (foto + nombre)

Mapa

Mis publicaciones / Mis favoritos privada

Logo · Tienda · Vender (foto) Nombre [Cerrar sesión]

Menú perfil

Título + [+ Nueva]

Card [Eliminar] | Card [Eliminar] | Card [Eliminar]

Card ♥ | Card ♥ | Card ♥

2. Navegación entre vistas

🔓 **Sección pública (sin sesión)**

Página principal `/` → **Regístrate** `/registro` **Iniciar sesión** `/login` · 404 ∗ también es pública

⬇ login exitoso: se guarda el token JWT en localStorage y el usuario en AuthContext ⬇

🔒 **Sección privada (requiere sesión, protegida por <PrivateRoute>)**

Tienda `/tienda` → **Detalle de publicación** `/publicacion/:id`

Perfil `/perfil` → **Mis publicaciones** `/perfil/publicaciones` **Mis favoritos** `/perfil/favoritos` **Nueva publicación** `/perfil/nueva`

Reglas de navegación

- Si un usuario **sin sesión** entra a una ruta privada, se le redirige a `/login` y, al iniciar sesión, vuelve a la página que quería ver.
- Si un usuario **con sesión** entra a `/login` o `/registro`, se le redirige a `/tienda`.
- El navbar cambia según la sesión: sin sesión muestra [Iniciar sesión] [Regístrate]; con sesión muestra Tienda, Vender, su foto/nombre y [Cerrar sesión].
- Datos de sesión en el frontend: `token` (localStorage) y `user = { id, name, email, picture }` (AuthContext). Los favoritos del usuario viven en FavoritesContext.

3. Dependencias del proyecto

Frontend (React + Vite)

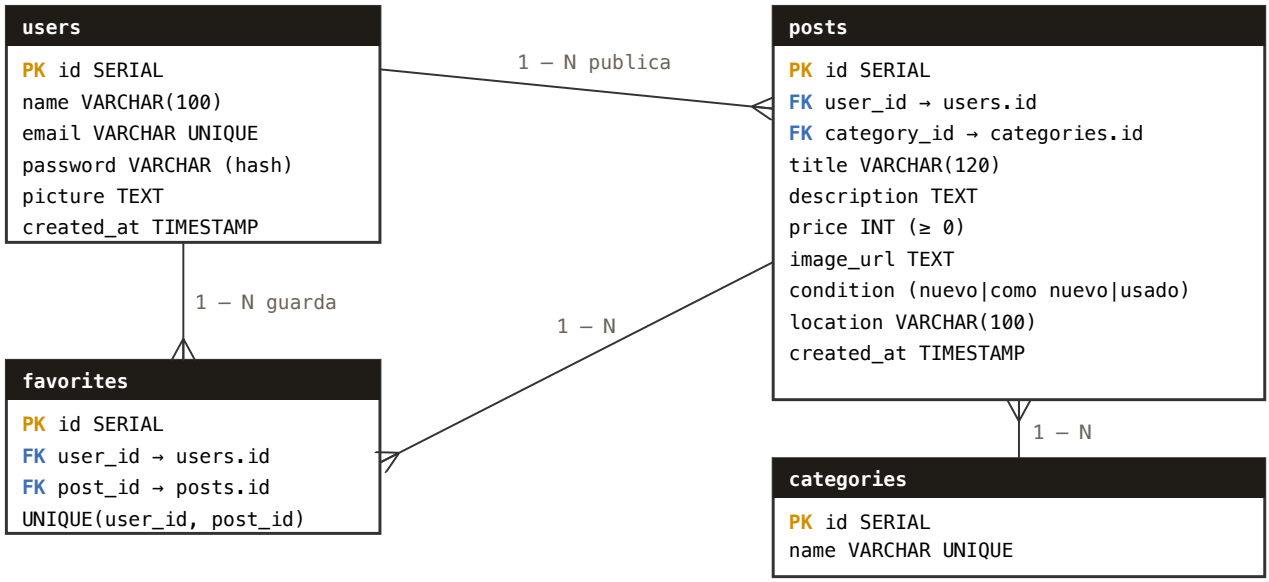
Paquete	Uso
`react`, `react-dom`	Librería de interfaz
`vite`	Creación del proyecto y servidor de desarrollo
`react-router-dom`	Navegación entre vistas, rutas privadas
`bootstrap`	Framework CSS
`react-bootstrap`	Componentes Bootstrap para React
`axios`	Consumo de la API REST, envío del token
`eslint` (dev)	Calidad de código

Backend (Node + Express)

Paquete	Uso
`express`	Servidor y rutas de la API REST
`pg`	Conexión y consultas a PostgreSQL
`jsonwebtoken`	Autenticación y autorización con JWT
`bcryptjs`	Encriptar contraseñas
`cors`	Permitir consultas de otros orígenes
`dotenv`	Variables de entorno (.env)
`jest`, `supertest` (dev)	Tests de rutas
`nodemon` (dev)	Reinicio automático en desarrollo

Base de datos: PostgreSQL. Despliegue: Netlify (frontend) y Render (backend + base de datos).

4. Tablas de la base de datos y relaciones



Relación	Tipo	Descripción
users → posts	1 a N	Un usuario publica muchos instrumentos; cada publicación tiene un único vendedor. `ON DELETE CASCADE`.
categories → posts	1 a N	Cada publicación pertenece a una categoría (Guitarras, Bajos, Baterías y percusión, Teclados y pianos, Cuerdas frotadas, Otros).
users ↔ posts (vía favorites)	N a N	Un usuario guarda muchas publicaciones como favoritas y una publicación puede ser favorita de muchos usuarios. La restricción `UNIQUE(user_id, post_id)` evita duplicados.

El script completo de creación está en `backend/db/script.sql` (incluido en este .zip).

5. Contrato de datos de la API REST

Base URL: `http://localhost:3000` (producción: URL de Render). Formato JSON. Las rutas con 🔒 requieren el header `Authorization: Bearer <token>`; sin token válido responden `401`. Los errores responden `{ message: String }`.

Objeto **Post** usado en las respuestas: `{ id: Number, title: String, description: String, price: Number, image_url: String, condition: String, location: String, created_at: Date, category_id: Number, category_name: String, seller_id: Number, seller_name: String }`

POST `/users` Registrar usuario

```
// request.body
{
  name: String,
  email: String,
  password: String,  // mín. 6
  picture: String    // opcional
}
```

```
// 201 Created
{ id: Number, name: String,
  email: String, picture: String }
// 400 faltan datos · 409 email ya existe
```

POST `/login` Iniciar sesión

```
// request.body
{ email: String, password: String }
```

```
// 200 OK
{ token: String,
  user: { id, name, email, picture } }
// 400 faltan datos · 401 credenciales inválidas
```

GET `/users/me` Datos del usuario logueado 🔒

```
// header Authorization: Bearer token
```

```
// 200 OK
{ id, name, email, picture, created_at }
```

GET `/users/me/posts` Mis publicaciones 🔒

```
// sin body
```

```
// 200 OK
[ Post, ... ]
```

GET `/categories` Listar categorías

```
// sin body
```

```
// 200 OK
[ { id: Number, name: String }, ... ]
```

GET `/posts` Galería con filtros

```
// query params (todos opcionales)
?search=String        // título o descripción
&category_id=Number
&order_by=recent|price_asc|price_desc|title_asc
&limit=Number&page=Number
```

```
// 200 OK
{ total: Number, page: Number,
  results: [ Post, ... ] }
```

GET `/posts/:id` Detalle de publicación

```
// params: id
```

```
// 200 OK
{ ...Post, seller_email: String,
  seller_picture: String }
// 404 no existe
```

POST `/posts` Crear publicación 🔒

```
// request.body
{ title: String, description: String,
  price: Number, image_url: String,
  category_id: Number,
  condition: String,   // opcional
  location: String }   // opcional
```

```
// 201 Created
Post
// 400 faltan datos · 401 sin token
```

PUT `/posts/:id` Editar publicación propia

```
// request.body: campos a cambiar
{ title?, description?, price?, ... }
```

```
// 200 OK  Post
// 403 no es el dueño · 404 no existe
```

DELETE `/posts/:id` Eliminar publicación propia

```
// params: id
```

```
// 204 No Content
// 403 no es el dueño · 404 no existe
```

GET `/favorites` Mis favoritos

```
// sin body
```

```
// 200 OK
[ Post, ... ]
```

POST `/favorites` Agregar a favoritos

```
// request.body
{ post_id: Number }
```

```
// 201 Created
{ id: Number, user_id: Number, post_id: Number }
// 400 falta post_id · 404 post no existe
```

DELETE `/favorites/:postId` Quitar de favoritos

```
// params: postId
```

```
// 204 No Content
// 404 no estaba en favoritos
```